Dyssonance enterprise AI challenge

Warrant

A gate between AI agents and the systems that move money.

Mithun M
Presenter, product and prototype

Thursday 5 November 2026

Before any agent moves money, Warrant returns one of three answers

Act
The evidence agrees and is current, so the agent goes ahead alone.

Gather
One fact is in doubt, so Warrant re-reads it before deciding.

Hold
The evidence conflicts, so one question goes to the person who owns the fact.

Warrant in three lines

The problem

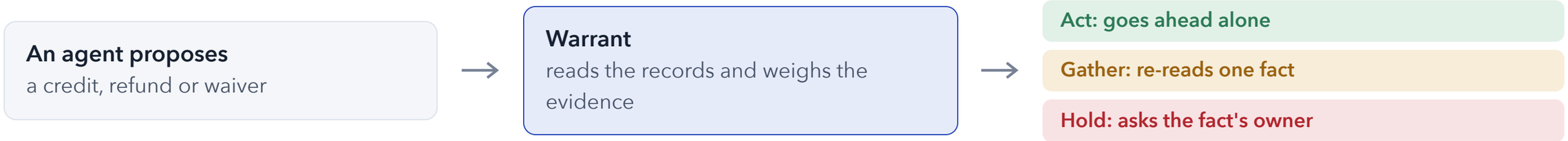
Support, success and retention agents now issue credits, refunds and waivers on evidence that can be out of date or contradicted by another system. At enterprises early in agentic adoption, the mistake can surface as late as month-end close.

The solution

Warrant sits in front of every money action. It reads the systems of record, weighs how current and consistent the evidence is, and returns act, gather or hold, with one question for the person who owns the fact.

Value and impact

Agents act alone when the evidence agrees, and people see only the real exceptions. On 224 generated test cases it stopped all 120 defects while sending about 7% of clean cases to a person on average (9 of 104 in the worst run).



Test results are on seeded, generated cases. Leakage saved in production is unmeasured: a log pull and a 30-day shadow pilot would size it

An agent can pass every check and still give away money it shouldn't

Seeded test case PL-05

An agent proposes a **300 USD** service credit: 5% of a 6,000 USD monthly fee, as the signed order form says.

- ✓ The data is fresh
- ✓ The amount is within the agent's authority
- ✓ There is no earlier credit for this incident
- ✓ It is under the contract's credit cap

✗ A signed amendment, effective 1 October, replaced the 5% rate with 3%. The right credit was **180 USD**, and nothing flagged the extra 120.

Why it happens

Each system is right about its own record. The order form, the amendment, the billing list and the risk flag live in different places, and no single check reads them together.

When agents disagree, mechanics settle it. The last write wins, or the most confident agent acts, and nobody records the view that lost.

Why now: the share of firms orchestrating multiple agents doubled in one quarter, from 9% to 18%.

KPMG Q2 2026 AI Pulse, 204 US leaders at firms above $1B revenue. Spawn CEO blog, 28 Oct 2026. How often this happens is unmeasured

Everyone sees a symptom; nobody sees the conflict behind it

Primary user

Support or applied AI engineer

Sees today: reversed credits and reopened tickets, never the conflict behind them.

Wants: to let agents do more without cleaning up after them.

Rules on holds

Billing or finance analyst

Sees today: credits that breach contract terms, sometimes found only at month-end close.

Wants: to be asked only when it really is an exception.

Account side

Customer success manager

Sees today: messages to the customer that contradict each other.

Wants: everything the account hears to agree.

Buyer

VP of Support or Chief Customer Officer

Sees today: automation rising with no view of the clawbacks against it.

Wants: to keep growing the agent budget without hidden rework.

> When two of my agents reach different conclusions about a customer, I want their actions to match what is true across our systems, so I can let them do more without cleaning up after them.

The engineer's job. The decision it changes: what agents may do alone, set per action type from evidence, not once in a prompt.

Size: TAM $3.0B and SOM $25M ARR by year 3, both assumed from 12,000 multi-agent enterprises at $250k a year. Rate and cost per incident are unmeasured

What primary users told me, and what public forums added

Three primary users in the segments Warrant serves. What each one described shaped the job on the previous slide.

CSM at an enterprise SaaS company

When the AI answered from one system and another held the truth, complaints went past the account team to the client's C-suite. A confident wrong answer cost more trust than a slow one.

Points to: actions that match what is true across systems

Support engineer at a Managed Service Provider (MSP)

Answers still sounded complete when a connector had broken or a decision had been superseded. Nobody could tell from the output.

Points to: silence must never mean safe

Business manager at a Fortune 500 insurer

When a plan changed mid-period, payouts were disputed over which version applied. The amended rule should govern only from its effective date.

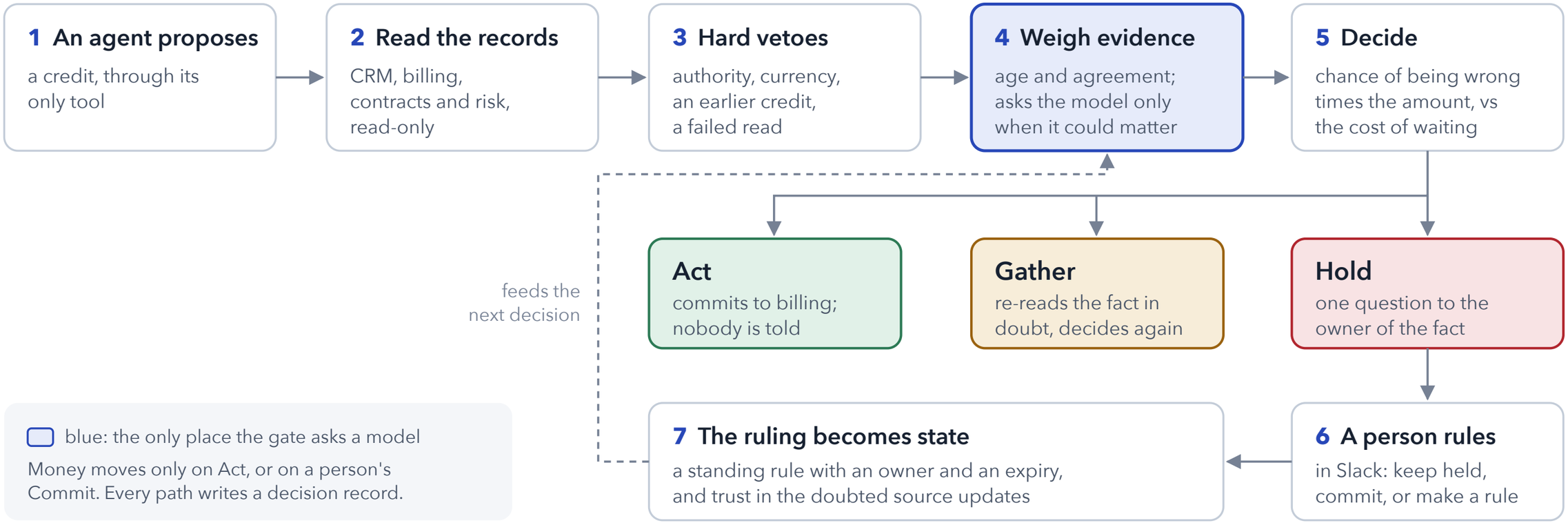
Points to: automation that does not quietly create leakage

Secondary research: sentiment analysis of 181 public opinions (163 Hacker News comments, 18 Substack posts)

Joint biggest gap: nobody can say which conclusion governs. **Loudest theme:** approval fatigue, 46 of 181. **What it overturned:** I assumed tracing was solved, but authors say logs cannot tell agent actions from human ones.

Primary research so far: three users. Next: 20+ interviews and 100+ online surveys. Forum authors are mostly developers, 1 of 181 read as finance, and a model scored each opinion, so this stands in for interviews

One credit, step by step: the model proposes, code decides



A fixed LangGraph pipeline, not an agent: no model chooses the gate's tools or path. Live on HubSpot and Chargebee test accounts; contracts and risk are local fixtures

A held credit reaches a person with the reason and one question

Held by the belief layer: one fact is too uncertain

Warrant APP

Held before commit. retention-agent proposes a service credit of **300 USD** on ACC-3007.

Why: P(safe) 0.021. Main doubt: the credit list may not yet show a recent write. Re-read in 285 s.

Question for the billing analyst: keep this held, or commit it?

Keep held | Commit as proposed

Held by a hard veto: the contract says no

Warrant APP

Held before commit. cs-agent proposes a service credit of **900 USD** on ACC-3008.

Why: above the contract's credit cap, 900 against 600 (10% of 6,000). Source: order form OF-3008, signed. Clause: "Service credits in any month may not exceed 10% of that month's subscription fee."

Question for the billing analyst: commit anyway?

Keep held | Commit as proposed

The person never opens four systems to find the conflict. The owner of the fact rules; the engineer owns the boundary of what agents may do alone.

I built the strongest harness I could, then found two problems it cannot solve

v1, 1 Nov

The gate

Held 10 of 10 planted defects. It also held 3 clean cases, and each time my test data was wrong, not the gate.

v2, 1 Nov

Precision

Fewer false holds and cheaper, but the median decision took 14.5 s. It failed the bar I had set, and I said so.

v3, 2 Nov

Live

One model, a live agent, real test systems and a real Slack card. 4.2 s per decision, 13 of 13 defects held.

v4, 2 Nov

Belief layer

Built for the two problems below. Its results follow the demo.

Ceiling one: the billing lag

A freshness window can only say "this record is recent". It cannot say "this list lags its own writes by over a minute". A freshly read list looked complete, and it was wrong.

Ceiling two: the flip

One clean case, CT-08, acted on some runs and held on others. The harness threw that disagreement away, so every act rested on the model finding nothing.

The trap: a missed conflict looks exactly like a clean case. Every version ran against a bar written before building; results in report section 15

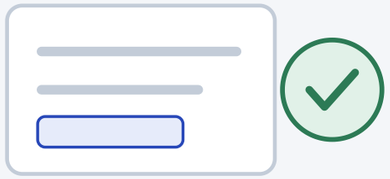
"Isn't this just a guardrail?"

A guardrail says yes or no. The belief layer also says how sure it is, and which fact would settle it.



Age, not a cliff

Evidence keeps its weight while fresh and fades near the old window, instead of dropping from trusted to useless at one hour mark.



Silence must be earned

Each check the model reports cites a record, and code confirms that record exists. Nothing found is a claim, not a default.



1 of 3 dissents: P(safe) 0.70

Disagreement is evidence

When the answer could change the decision, the model runs three times. One dissent lowers confidence instead of being thrown away.



Expected loss decides

Chance of being wrong times the amount, against the cost of waiting: act, re-read or hold. Hard vetoes still win every time.

Why it exists. v3 could only say fresh or stale, and treated model silence as safe. It could not express a credit list that lags writes, or a model that flips between runs.

The Bayesian part. A source starts 0.995 reliable, and one kept hold drops it to 0.990. Ageing: a billing read counts 0.995 fresh, 0.90 at its 6 h window, 0.81 at 12 h.

P(safe) on a $300 credit. Multiply the facts. A $20 cost of delay tolerates 6.7% doubt. All clean and fresh: 0.975, so it acts. 40 s after a billing change: 0.35, so it re-reads.

P(safe) assumes independent facts. All priors are assumed and were fixed before the runs. The unread-fact value of 0.8 is a policy, not a measurement

Live: a real agent, real test systems, a real click

1. **A clean credit** commits to Chargebee, and nobody is told.
2. **A second agent credits the same incident**, with the ledger switched off. The belief layer gathers on its own, showing P(safe) and when it will re-read.
3. **Keep held in Slack.** One click lowers how much Warrant trusts billing's credit list.
4. **Run it again.** The next clean credit shows the lower P(safe).
5. **The prototype screens:** the autonomy boundary, the shadow report and the review.

Real

The agent, the gate, reads from HubSpot and Chargebee test accounts, the credit write, and the Slack clicks.

Scripted or local

The tickets, the seeded accounts, the contracts and the risk flags.

DEMO

Seeded, fictional accounts only. A labelled replay of a recorded run is ready if the live path fails

Every eval passed. Then the live agent paid a customer twice



Schematic, not to scale

Why the evals missed it. A test fixture is always current, so this failure could not appear in any test I had written.

The fix was not a better model. The gate keeps a ledger of what it committed, and stops when the balance shows more credit than the list does.

The thesis applies to the gate too. A belief can be well formed, freshly read, and still wrong.

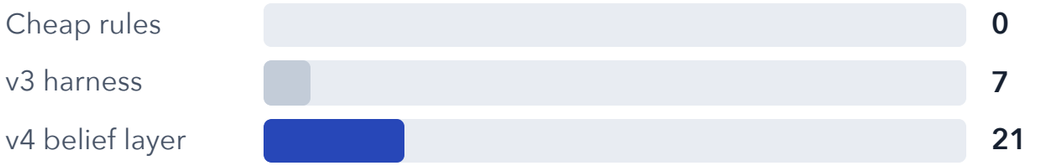
Logged in product/llm-failure-modes-log.md with 34 other failures from this build. The second agent in this run was on a test account

All 120 generated defects stopped, with two misses against my bar

Defects stopped, of 120 (worst of 5 runs)



Clean cases stopped, of 104

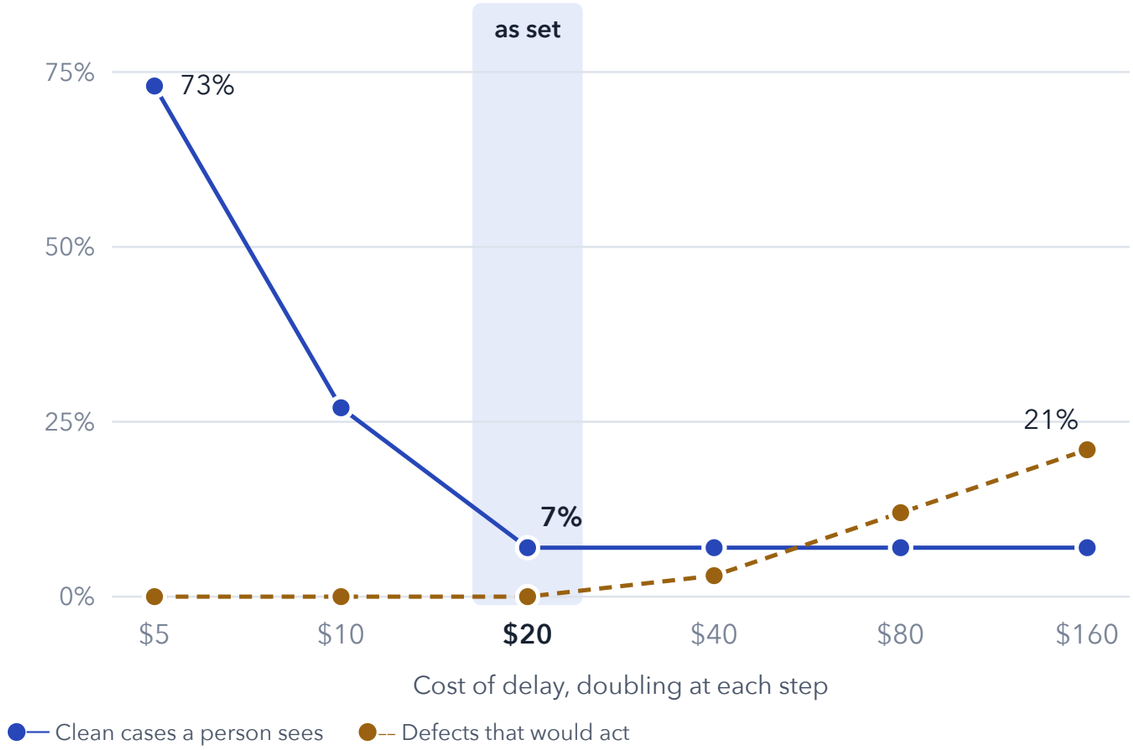


12 of v4's 21 only wait for a timed re-read; no person is involved. So 9 of 104 reach a person in the worst run, about 7% on average.

Also measured	v3	v4
Clean fixtures held	1 of 11	1 of 11
Cost per decision	$0.0101	$0.0266
Latency, median	4.2 s	4.5 s
CT-08 same on all 5 runs	No	**No: misses the bar**
Calibration within 0.10	Not measured	**3 of 6 bins off: misses**

224 cases generated by code, so the labels do not come from a model. About half are defects, unlike production. Lag cases pass partly by design, since τ came from the same lag

At a $20 cost of delay, a person sees 7% of clean cases and no defect acts



$20 is not the cost of a run. It is the buyer's dial: what making one case wait costs, $2.67 of review plus $17.33 assumed.

Below $20, no defect acts, but a person sees up to 73% of clean cases. **Above $20 is worse:** defects act and review does not fall. So $20 is the best point tested.

Saved review time alone does not pay for this. Reviewing all 5,000 cases a month (assumed) costs $10,000 to $13,333; at $20, $1,170 to $1,570. That saves about $106k to $141k a year, below an assumed $250k price.

The case rests on what is not yet counted: clawbacks, lost trust and manual account work.

Stored beliefs replayed at each cost, with no new model calls. Excludes timed re-reads. A measured 2 to 2.67 USD per human review, at an assumed $30 to $40 an hour. Cost table in the appendix

What I chose, what I gave up, and what would change my mind

Chose	Gave up	Switch if
A belief layer, not a bigger model	Cost: 2.6 times per decision	A single sample holds the catch rate
Labels from how cases were built, not from a model	Realism: half the set is defects	A log pull gives real cases
Priors frozen before the runs, even the two now known wrong	A better score	New cases, unseen by these runs, exist to re-tune them on
Fail closed, shadow first: hold if a check or source fails, and score real tickets without writing before any autonomy	Automation rate	Over-holds cost more than the leakage they stop

The two priors now known wrong: the credit list's 90 s т, and the unread-fact 0.8, a policy that was never going to calibrate

Usage moves first; credit leakage decides whether it worked

Key metrics	Leading: moves first	Lagging: decides if it worked	Counter: rises if we win the wrong way
Business	**Shadow money:** value of held actions finance confirms were wrong **Autonomy widened:** action types promoted from shadow or approval	**Credit leakage** per 1,000 money-action tickets **Cost per resolution,** with review time **Retention** of affected accounts	**Hold cost against leakage stopped:** delay and review spend per dollar saved
Product	**Coverage:** money actions through the gate, not around it **Time to ruling:** median, and share inside SLA **Rule reuse:** contested cases settled by an existing rule	**Job done:** tickets closed with no reversal, reopen or contradiction in 30 days	**Rubber-stamp rate:** holds ruled "act as proposed", split by whether those actions are later reversed
Technical	**Hold precision:** holds where the ruling differed from the proposal **Defects stopped in evals.** Today: 120 of 120.	**Calibration on real outcomes:** stated confidence against what happened. Today: 3 of 6 bins off.	**Over-holds:** clean cases reaching a person. Today: about 7%. **Cost and latency per decision.** Today: $0.0266, 4.5 s median.

No production data yet: every "Today" is measured in evals on generated cases, and there are no targets until a log pull sets a baseline. Job done is leading for finance's numbers; it is lagging here because it takes 30 days

Three ways I could be wrong

1

The evidence base is still small

It rests on minimal primary research, my prior experience in this segment, and sentiment analysis on a small corpus of forum opinions. Targeted JTBD interviews with a larger pool of potential customers, and a much larger forum corpus, would give a basis to trust.

2

The results partly grade themselves

Sonnet 5.5 runs the agent and the compare step; Opus 5.5 is the eval judge. The belief layer and the 224 generated cases are code, not a model, but I wrote that code with Claude Code. My review of 15 records is the first outside check, by the author: 11 matched v3 and 10 matched v4.

3

The live path is real but small

Scripted tickets, seeded accounts and one kind of write. A cited record shows it exists, not that the model read it, and P(safe) is not yet a calibrated probability.

Each has a named test: interviews and surveys at scale, labels from people who did not build it, and a shadow pilot on real traffic.

What I would do next

The next 90 days

1. **20+ interviews, 100+ online surveys and one log pull.** How often do agents act on conflicting sources, and can finance tie it to leakage it already sees?
2. **Design partners in mid-market SaaS and MSPs**, where one leader owns support and success. Enterprise is the expansion market.
3. **A 30-day shadow pilot.** It worked if finance confirms a meaningful share of held money as real leakage, and the engineer widens autonomy on that evidence, with clean actions held at 5% or less assumed.

How it grows

The shadow report is the sales artifact. It sizes a prospect's own held money before they buy anything.

Every ruling is product signal. It becomes a standing rule, an eval case and a change in how much a source is trusted.

Integration point: a stateful reasoning engine would replace belief.py, behind the same hard vetoes.

With half the time, I would have cut the boundary screen, the shadow report and the forum evidence, and kept the live path

I journaled every judgment call, and the same failure shows up in two more industries

The build journal, in the public repo

Every checkpoint is a commit: 80+ commits, and one pull request per version, each with its bar, results and misses.

Every judgment call is written down: a memo at each fork, from discovery to the v4 plan and the reasoning behind it.

Every failure is logged with evidence: 35 so far, across build, product and evals, and each lesson becomes a rule I reuse.

The seed for Warrant in product and build work. My own build agent showed the same failure: an instruction hardened in its memory and nearly led to a history rewrite. The journal is labelled data for a gate on product and build agents.

The next segments I considered at the start

On-call, DevOps and IT support (RMM)

An auto-remediation agent restarts a database because a monitoring alert says it is down. The alert is past its 30-minute window, and the stop was a planned migration.

Synthetic corpus and planted cases are in this repo; not yet run through the gate.

Logistics and freight

The customs broker says a container is on hold; the carrier says it is released. In my freight exceptions copilot, a guard that abstains and names the field in doubt took end-to-end accuracy from 0.857 to 1.00 on 7 exceptions.

github.com/superposition-fc23/freight-exceptions-copilot

Journal: product/build-learnings.md, product/llm-failure-modes-log.md and one pull request per version, at github.com/superposition-fc23/Warrant_system-LLM_Failure_modes

Thank you

Questions?

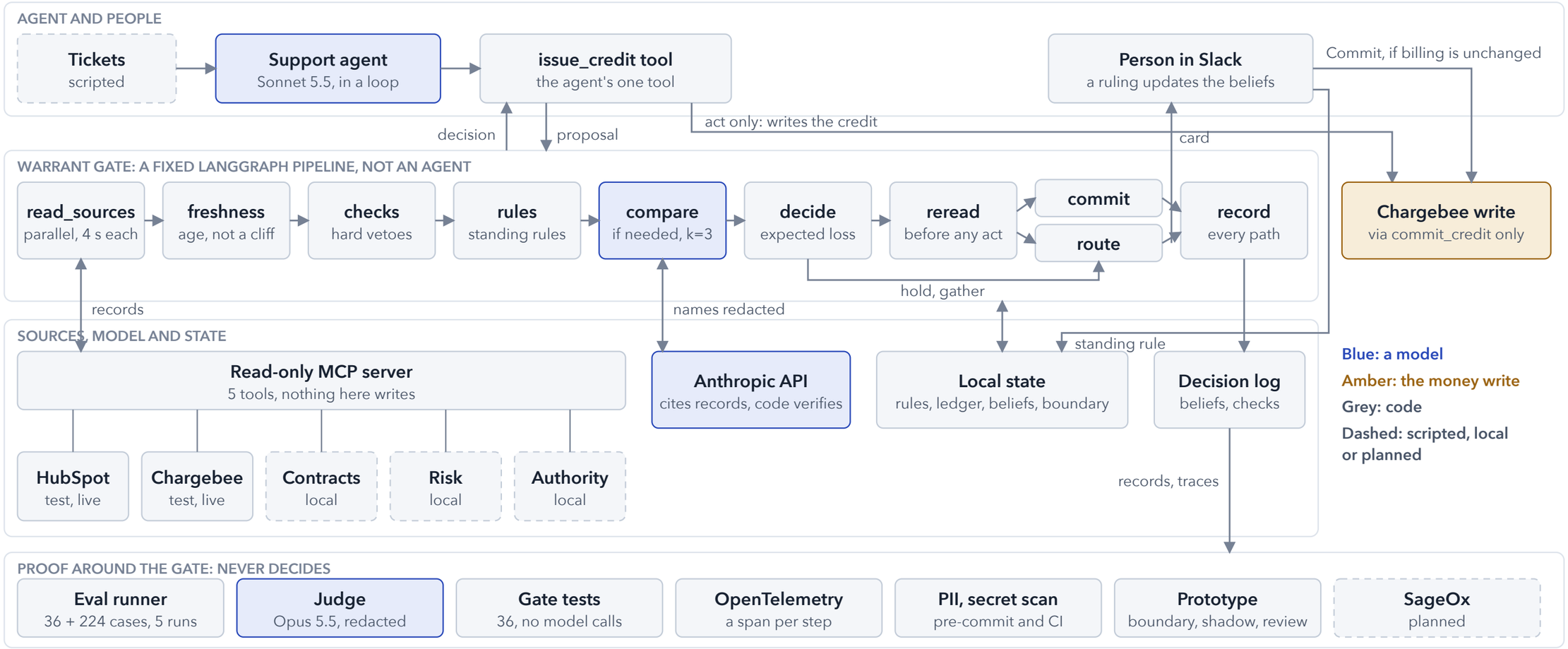
Mithun M

Code, run logs and the full report: https://github.com/superposition-fc23/Warrant_system-LLM_Failure_modes

What it must not do

- **Never act on a failed read.** Timeout, error or malformed reply gathers. Tested
- **Never move money without an act** on that exact proposal, re-read before commit. Tested
- **Never settle a disagreement silently.** The losing belief stays on the record. Built
- **Never apply a rule outside its scope** or past its expiry. Tested
- **N3, kill switch:** reversal rate 1 point above baseline over 7 days, or one reversal the gate should have caught, reverts that action type to approval and tells the VP and engineer within 15 minutes assumed Not built

The system as built in v4



Test accounts and seeded data. Nothing yet stands in front of a production action

About $2 to $2.67 a record, against $0.01

Cases, 10,000 customers assumed	A day	A week	A month	Review, a month	Review, a year
0.2 per customer a month	67	467	2,000	$4,000 to $5,333	$48k to $64k
0.5 per customer a month	167	1,167	5,000	$10,000 to $13,333	$120k to $160k
1 per customer a month	333	2,333	10,000	$20,000 to $26,667	$240k to $320k

- **Measured once:** 15 records took me about an hour, 4 minutes each. At $30 to $40 an hour assumed, that is the figure above
- **Review time alone does not pay for this.** The dial slide shows why the case rests on what is not counted

One reviewer, 15 synthetic records. Every case reviewed; cost falls in line with the share a person sees

What a belief layer replaced, and what it did not

Harness part	What v4 showed	What stays regardless
Freshness windows	Decay caught the lag and aged reads. Hand-set, not learned	A floor set by policy
Hold when the model reports a conflict	Stopped more defects. P(safe) missed the calibration bar	A threshold set by the cost of being wrong
Recompute every case	27% took no model call, against 16%. Cost still rose	Reading the systems of record
Standing rules	A ruling moves source reliability. Partly shown	A named owner who can overturn one
Authority map, code checks, evals	Untouched	People with authority; independent evaluation

Still untested: a stream of evidence updates, which is what a true stateful engine would be for